

LocalHouseLLM Research

Independent AI Systems & Local Inference Research Division

AI Optimization for Local Inference: Highest-Impact Layers

Report 06 · Five-Layer Execution Stack Analysis | Prepared by LocalHouseLLM Research | Confidential Technical Compendium

AI Optimization for Local Inference: The Highest-Impact Layers

LocalHouseLLM Research

Executive Summary

Local LLM inference hinges on squeezing maximum performance from the execution layer. Across five key subsystems — speculative decoding, attention kernels, scheduling/batching, compilers, and GPU memory/exec — researchers and engineers have devised numerous tricks. Some yield dramatic speedups (often 2–6x or more) by restructuring computation or data flow, while others eke out smaller gains by optimizing memory use or launch overhead. Speculative decoding techniques (e.g. Leviathan et al.’s draft-and-verify) can ~2–3x throughput by running small “draft” models in parallel with the big model. Attention kernel work (FlashAttention variants) fuses operations to slash memory traffic, yielding order-of-magnitude latency drops on large contexts. Advanced scheduling/batching (vLLM’s continuous batching, PagedAttention) boosts GPU utilization by ~3–4x versus naive batching. Compilers and IR frameworks (TorchInductor, AI Template, TensorRT, etc.) automate fusion and lowering, often doubling performance via kernel fusion and autotuning. Low-level GPU memory tricks (paged KV caches, async copies, CUDA Graphs, stream overlap) cut idle time and reduce memory overhead, directly improving latency and throughput.

Across all five layers, the highest-leverage wins collapse multiple operations or memory movements into single kernels and keep the GPU fed. Kernel fusion and batch scheduling have immediate large impacts and are now production-ready in many frameworks. Very new research (cluster-level fusion, diffusion-based speculative decoding) shows great promise but remains experimental or niche. Typically, the fastest modern techniques are also the most complex, while simpler approaches (static batching, compiler flags) are easiest to deploy but give smaller gains. By layering these optimizations, current systems have cut inference latencies by multiple factors (up to ~10x in some cases) while preserving exact output quality.

Historical Evolution of Tier-One Inference Optimizations

Speculative Decoding

Speculative Decoding was introduced (Leviathan et al., 2023): a small draft model predicts many tokens in parallel, and the large target model verifies them in bulk. Speculative Sampling and Speculative Prefix Sampling (Chen et al. 2023) studied acceptance rates. In 2023–24, Draft & Verify (Zhang et al., ACL 2024) showed the same LLM can be used as both draft and target by skipping layers (self-speculation). Meta’s LayerSkip (2024) trained LLMs with “early exit” to allow full self-speculation. Other 2024 advances included Lookahead Decoding (Fu et al., 2024), a Jacobi-style predictor achieving ~1.8–4x speedups with multiple GPUs. Diffusion-inspired methods appeared in 2024–25 (DART, SpecDiffusion), where a diffusion model drafts multi-token sequences, achieving up to 7x standard decoding throughput. Tree-based speculative schemes like JetSpec (2026) and Recursive SpecDecoding (2024) explored branching draft generation. The latest (2025–26) focuses on verification algorithms for tree drafts (Traversal Verification).

Attention/Transformer Kernel Optimization

The FlashAttention breakthrough (Dao et al., NeurIPS 2022) fused all attention steps in one kernel, immediately giving ~5–9x speedups on GPUs and linear memory scaling. FlashAttention-2 (ICLR 2024) parallelized along sequence-length; FlashAttention-3 (2024) optimized for Hopper (H100) with warp specialization and asynchronous pipelines. FlashAttention-4 (2026) targeted Blackwell GPUs, yielding up to 1.3x over cuDNN and 2.7x over prior Triton kernels. Sparse Attention research led to kernel-level work: NSA kernels and Flash-Sparse-Attention (ICLR 2026) re-implement sparse patterns, achieving ~1.6x kernel speedup. PagedAttention (Kwon et al., SOSP 2023) applied OS-style paging of KV cache, enabling 2–4x higher throughput in vLLM. Other innovations include block-sparse/local-window kernels, kernel fusion for downstream ops (ClusterFusion fusing QKV+Attn+Projection into one GPU pass), and custom CUDA/Triton kernels.

Runtime Scheduling and Batching

Originally, inference systems did static batching. Dynamic batching (Triton Inference Server, TF Serving) grouped queries arriving close in time. In 2023–24, continuous/dynamic batching took off: vLLM introduced in-flight token batching where new requests join as soon as capacity frees, pumping near-saturation throughput. vLLM’s scheduler also pioneered chunked prefill. FlexLLM (2024) studied token-level scheduling where tokens are scheduled independently across requests. Fairness and priority scheduling (Google’s Duet system) remain mostly custom. Pipeline balancing became crucial: splitting API server from GPU process and pre-computing multiple steps before CPU sync gave 28% throughput gains. CUDA streams and CUDA Graphs cut launch overhead (often automatic in compilers).

Compiler/IR/Graph Optimization

Torch.compile/TorchInductor (PyTorch 2.0) has become a de facto baseline, tracing FX graphs via TorchDynamo, fusing pointwise ops, and lowering to Triton/C++ with autotuning — routinely yielding ~1.8–2x speedups. Meta’s AITemplate (2022) transforms models into template C++/CUDA code, reporting up to 12x speedups vs PyTorch eager on A100. Google’s OpenXLA compiles to optimized kernels, especially for TPU. TensorRT and ONNX Runtime serve as other compiler backends. MLIR-based projects (Torch-MLIR, IREE, TVM) perform graph rewrites, constant folding, dead-code elimination, and layout transformations. Torch.compile is easiest to deploy (one decorator) and robust; TVM/LLVM-based systems offer finer control but require more engineering.

GPU Memory Hierarchy and Execution Optimization

The lowest level of tuning: HBM/DRAM optimization (PagedAttention avoids fragmentation and redundant KV copies); shared memory/registers (custom kernels keep intermediate activations on-chip); memory pooling/allocation (reusing scratch space to reduce cudaMalloc/free calls); pinning and zero-copy for faster CPU→GPU writes; async copies and stream overlap to hide latency; NUMA awareness in multi-GPU servers; L1/L2 caching hints; launch overhead reduction via CUDA Graphs and persistent thread blocks; occupancy tuning; Tensor Core utilization via FP16/FP8/structured sparsity. These memory/workload tricks typically yield tens-of-percent improvement individually, but in aggregate unlock the gains from higher layers.

Complete Taxonomy of Methods

- **Speculative Decoding:** Standard SpecDec (Leviathan 2023), Speculative Sampling (Chen 2023), Draft&Verify (Zhang 2023), LayerSkip (Meta 2024), CLaSp (Chen 2025), SpecPV (Tan 2025), Lookahead (Fu 2024), N-Grammys (Talebi 2024), DART (Liu 2024), SpecDiffusion (Christopher 2025), RSD (Jeon 2024), JetSpec (Hao 2026), Traversal Verify (Lou 2025).
- **Attention/Transformer Kernels:** FlashAttention v1–4 (Dao 2022–26), FlashInfer (Ye 2025), FlashSparse/FSA (Yan 2026), SageAttention (Shen 2024), AITemplate (Meta 2022), Triton kernels, BlockSparse (FlashInfer/PagedAttention), ClusterFusion (Lu 2025), BigBird/Local windows with optimized kernels.
- **Scheduling/Batching:** Static batching, Dynamic batching, Continuous batching (vLLM 2023+), Token-level scheduling (FlexLLM 2024), Priority/fairness schedulers, Chunked prefill, Multi-tenant scheduling, Overlap compute/transfer, CUDA Graph pipelining, Pipeline parallelism.
- **Compiler/Graph:** Torch.compile/Inductor (2023), TorchScript, OpenXLA/XLA, TVM, IREE, Glow, AITemplate, TensorRT, ONNX/ORT, PopTorch/NNEF, MLIR/HLO pipelines.
- **GPU Memory/Execution:** Paged KV Cache (vLLM 2023), CUDA Graphs, pinned buffers, async memcopy, NVLink collective ops, shared-memory KV, memory pooling, persistent kernels, stream concurrency, occupancy tuners, warp specialization, thread block clustering (ClusterFusion 2025).

Detailed Technical Entries

Speculative Decoding Methods

Standard Speculative Decoding (Leviathan et al. 2023): Uses a small, fast draft model to generate a sequence of tokens, then runs the large target model in parallel on that prefix, comparing probabilities to accept or reject each token — guaranteeing the exact same output distribution as standard decoding. Performance: typically ~2–3x throughput, reduces memory traffic by ~30–40% when acceptance is high, no loss of quality. Limitations: gains depend on draft quality and acceptance rate; adds complexity from two-model orchestration. Stability: usable but niche. Obscurity: known to specialists.

Speculative Sampling (Chen et al., 2023): Improves practical acceptance rate via tempered sampling that maximizes acceptance chance, derived from theoretical analysis. More an analysis contribution than standalone method. Stability: research prototype. Obscurity: niche.

Self-Speculative Decoding (Draft & Verify, Zhang et al. 2023): Avoids needing a separate draft model by “skipping” a batch of intermediate transformer layers in the same model to generate draft tokens, then running all layers for verification. Up to ~1.99x speedup on LLaMA-2, no accuracy change. Readiness: production-proven, open-source, simple to integrate. Stability: stable. Obscurity: known to specialists.

LayerSkip (Meta, 2024): Trains the model with random layer-dropout and an “exit loss” during training so a truncated model learns to predict well, enabling stronger self-speculative acceptance at inference. Speedups of 1.34x–2.16x on summarization/coding tasks, near-identical output quality. Requires costly retraining. Stability: usable but niche. Obscurity: niche.

CLaSp (Chen et al., ACL 2025): Dynamic-programming-based adaptive layer-skipping per token, choosing how many layers to skip based on context. 1.3–1.7x speedup on LLaMA-3 variants, <0.5% output change. Stability: experimental prototype. Obscurity: hidden gem.

Speculative Diffusion (SpecDiff, Christopher et al., NAACL 2025): Uses a discrete diffusion model to non-autoregressively sample an entire block of future tokens, verified via standard decoding — a “speculative block” rather than a chain. Up to 7.2x faster than standard decoding, ~1.75x faster than prior spec-decoding methods. Status: research prototype, no widely available code. Obscurity: niche.

DART (Liu et al., ICML 2024): Generates multiple masked tokens in parallel using diffusion-like ideas (Gaussian noise on logits), building a token tree with N-gram constraints. 2.03–3.44x speedup, ~30% better than EAGLE3 baseline. Obscurity: known to specialists.

Lookahead Decoding (Fu et al., 2024): Independently generates candidate tokens for future timesteps and verifies them in parallel using a Jacobi-style update, with no extra training. ~1.8x speedup on conversation tasks, ~4x on codegen with 8 GPUs. Status: research prototype.

JetSpec (Hao et al., 2026): Trains an LLM with a special “tree-causal” attention that generates a full tree of next-token hypotheses in one pass, then verifies via target model. Up to 9.64x on math problems and 4.58x on chat tasks (~1000 tokens/s on a single GPU). Status: experimental/hidden.

Traversal Verification (Lou et al., 2025): In a multi-candidate draft tree, verifies from leaves up (bottom-up) rather than top-down, preserving partial sequences standard verify would drop. 2–6% improvement in acceptance length over conventional verify, exact output distribution guaranteed. Status: research prototype.

Comparisons: Multi-token diffusive methods (SpecDiff, JetSpec) achieve the highest raw speedups but are least mature. Standard spec-decoding is easiest to implement and robust (lossless), gains modest (~2x). Self-speculative (Draft&Verify, LayerSkip) trades minimal retraining for simpler deployment. Lookahead and CLaSp occupy the middle ground. Production systems mostly use standard draft methods since they work on any model without retraining.

Attention / Transformer Kernel Optimization

FlashAttention (Dao et al., NeurIPS 2022): Fuses all attention steps in one kernel with tiled execution, eliminating intermediate writes and making memory usage linear instead of quadratic in sequence length (~10x memory saving at 2K tokens, ~20x at 4K). Up to ~5–9x throughput vs naive matmul-based attention. Stability: production-proven. Obscurity: mainstream.

FlashAttention-2 (Dao, ICLR 2024): Parallelizes over sequence length, allowing multiple thread-blocks to cooperate on one attention call, improving GPU occupancy on large sequences — roughly 2x faster than FlashAttention-1 on certain shapes. Stability: stable. Obscurity: mainstream.

FlashAttention-3 (Shah et al., SYSML 2024): On Hopper GPUs, uses asynchronous (2-CTA) scheduling, specialized warp kernels, and FP8 support — roughly 2x faster than cuBLAS on H100. Stability: stable. Obscurity: mainstream among H100 users.

FlashAttention-4 (Zadouri et al., arXiv 2026): For Blackwell GPUs (B200/GB200), pipelines two MMA per warp, uses fast software math, and larger tile sizes to fully occupy tensor cores. Up to 2.7x faster than Triton and 1.3x over cuDNN 9.13 on BF16, achieving ~71% of peak (1613 TFLOPs on B200). Status: cutting-edge research (2026). Obscurity: hidden gem.

SageAttention Family (Shen et al., 2024-25): Quantized attention kernels (INT8/FP8/INT4/FP4) for consumer GPUs, yielding 1.5–2x speedups by lowering precision. Status: research prototypes, obscurity: deep underground (mostly consumer GPU focus).

FlashInfer (Ye et al., MLSys 2025): Provides highly-optimized kernels plus JIT-generated fusions, block-sparse KV cache format, and dynamic scheduling. 29–69% reduction in inter-token latency vs prior serving compilers; 28–30% better long-context latency; 13–17% faster in parallel-generation. Status: production-ready, integrated into popular inference servers. Stability: stable. Obscurity: known to practitioners.

Paged (Key-Value) Attention (Kwon et al., SOSP 2023): Treats GPU memory like OS “pages,” allocating KV cache in fixed-size blocks that can be shared and swapped across sequences, eliminating fragmentation and duplication. Enables 2–4x throughput improvement over FasterTransformer/Orca at same latency, reduces GPU memory use ~20%. Status: production, core of vLLM engine. Stability: stable. Obscurity: mainstream in inference.

Flash-Sparse / FSA (Yan et al., ICLR 2026): Reorders loops in the kernel to better utilize varying head counts (small GQA groups) on modern GPUs. Up to 3.5x faster kernels than NSA baseline, ~1.25x end-to-end training speed, ~1.36x faster prefill decode. Stability: research prototype. Obscurity: niche.

Flash-Sparse-Attn Library (Shi et al., HKUSTDial): Trainable threshold/gating (drop keys below threshold), local windows, grouped Q attention on top of FlashAttn fusion, handling extremely long contexts. Stability: usable but niche. Obscurity: known to specialists.

ClusterFusion (Luo et al., NeurIPS 2025): Uses NVIDIA Hopper’s cluster shared memory with new primitives (ClusterReduce/Gather) to fuse QKV projection + attention + output-projection in one mega-kernel, eliminating global memory writes between layers. ~1.61x end-to-end latency speedup over best existing frameworks on H100. Stability: experimental. Obscurity: hidden gem.

Comparisons: FlashAttention is easiest to use and usually very fast on NVIDIA GPUs. FlashInfer adds scheduling for an extra 20–50% speedup. AiTemplate is very powerful but requires separate compilation. Sparse kernels help only models explicitly using sparsity. PagedAttention is more about batching/memory. For most LLMs, FlashAttention + memory optimizations gives the best performance today; exotic fusions like ClusterFusion are promising but not mainstream.

Scheduling and Batching Methods

Static Batching: Pre-groups incoming requests by similar input lengths. Simple, stable p50 latency at full utilization; low GPU utilization at low QPS, inflexible to variable arrivals. Stability: stable, production-proven. Obscurity: mainstream.

Dynamic Batching (e.g. Triton): Maintains a queue; forms a batch when enough requests arrive or a timeout hits, padding shorter ones. Higher throughput than static at moderate load; head-of-line blocking can occur. Stability: stable. Obscurity: mainstream.

Continuous/Dynamic Batching (vLLM's approach): Batches continuously — as soon as any slot frees, a new request slots in without resetting the GPU; pre-computes multiple decoding steps in one GPU run (multi-step scheduling) to amortize CPU costs. Achieved ~3.7x higher token throughput than HF's static API in tests, p50 latency 1.5-2x lower at high load. Status: production-proven (vLLM, Orca, MLC). Stability: stable. Obscurity: known to specialists.

Token-Level Scheduling (FlexLLM, Aegaeon): Pools tokens across all requests rather than grouping requests, for very high GPU utilization, but hard to implement (needs "jumping ahead" token support). Status: research prototype. Obscurity: deep underground.

Chunked Prefill (vLLM): Encodes a prompt in chunks so decoding can start earlier and fill the GPU with new tokens, reducing time-to-first-token for long prompts. Status: production. Obscurity: specialist.

Overlapping Compute and Transfer: Uses CUDA streams and pinned memory to copy data concurrently with GPU compute; asynchronous copying keeps the GPU from waiting on host. Can improve throughput by ~5-15% by reducing idle gaps. Stability: production. Obscurity: mainstream concept.

CUDA Graphs and Persistent Kernels: Captures repeated kernel-launch sequences as a CUDA Graph, or uses one persistent kernel that loops to decode many tokens, eliminating launch latency. Reported improvements ~10-20% in end-to-end LLM batching. Status: production (PyTorch 2.x, TensorRT).

Fairness and Priority Scheduling: Assigns weights/priorities to requests, or caps max tokens per request to prevent starvation. Mainly software overhead, reduces tail latency for critical users. Status: implementation-specific.

Comparisons: Continuous batching (vLLM) yields the highest throughput and GPU utilization. Static batching is easiest to deploy but worst on p95 latency. Token-level and overlapping strategies eke out further gains but are complex. Smart chunking and asynchronous graphing are the most overlooked, exploited by few mainstream toolkits by default.

Compiler / IR / Graph Optimization

Torch.compile (TorchInductor): Traces and optimizes graphs via FX, fusing pointwise ops, autotuning block sizes, choosing best GEMM path, and generating Triton/C++ kernels. Reported ~1.8-2x speedups on standard models; user tests show 10-50% per-layer improvement, up to 3.8x for small fused subgraphs. Production readiness: production-proven (PyTorch default in vLLM, Meta's Llama port). Limitations: long warm-up compile time; not all patterns fuse perfectly. Stability: stable. Obscurity: mainstream.

TensorRT (NVIDIA): Imports via ONNX/UFF, optimizes graph (layer merging, kernel selection, precision calibration), runs optimized engines. Often ~2x speedup for supported models (especially INT8). Production: widely used but integration can be heavy (C++ APIs). Stability: stable, production-proven.

OpenXLA / TPU/MLIR (Google): Compiler lowering to HLO/MLIR, originally for TPUs but expanding to CPU/GPU. On GPU, can compete with other frameworks (1-2x). Status: usable on JAX/Flax, emerging for PyTorch.

AI Template (Meta, 2022): Transforms PyTorch models into fused C++ template kernels via vertical/horizontal/memory fusion, relying on FlashAttention for attention. Meta reports up to 12x faster than PyTorch eager on A100 for CNNs and transformers (likely peak-case). Ease: requires exporting model to AIT's format, not drop-in. Stability: stable. Obscurity: known to specialists.

TVM (Apache): Defines compute graph, applies schedule primitives, runs autotuner for best config per hardware; codegen to CUDA/LLVM. Achieves 1.5-2x PyTorch speed with careful tuning. Stability: usable but niche.

IREE (Google): MLIR-based runtime for CPU/GPU, lowering PyTorch/XLA models to MLIR for JIT or AOT compilation. Promising for portable inference, not widely adopted in LLM yet. Stability: experimental.

Generic compiler techniques: Fusion (merging op chains to reduce memory trips), layout transform, autotuning (best GEMM kernel selection), shape specialization (fixed batch/seq sizes), memory planning, dead-code elimination, constant folding.

Comparisons: Torch.compile is easiest (push-button, auto fusion), reliably yielding 1.5-2x. AI Template can yield higher peak (up to 12x in some reports) but needs more engineering. MLIR/TVM/IREE need more setup but allow finer tuning. The most overlooked areas are schedule autotuning and graph partitioning for heterogeneous hardware.

GPU Memory Hierarchy and Execution Tricks

Shared Memory / Register Usage: Custom kernels load Q/K/V fragments into shared memory before compute (FlashAttention-style), keeping intermediate softmax outputs on-chip. Fusing 3 kernels into one changed 5 DRAM round-trips into 2, speeding a Transformer layer by ~3.8x in one teardown. Stability: mainstream practice.

Kernel Launch Reduction (CUDA Graphs / Persistent Kernels): Groups many kernel launches into a graph, or uses a single looping kernel. PyTorch 2.x/TorchInductor auto-applies CUDA Graphs when possible. Multi-step scheduling improved vLLM throughput by 28% on 70B x 4H100 partly via reduced overhead.

CUDA Streams and Concurrency: Uses multiple CUDA streams to overlap memory copies and kernel execution. Reports of continuous overlap improving utilization by 8.7% in one vLLM case.

Occupancy and Warp Specialization: Adjusts thread/block config for high occupancy; warp specialization lets some warps handle shared-memory ops (FlashAttention-3's approach). Can raise occupancy from ~50% to 90% on new architectures.

PCIe/NVLink Optimization: Overlaps GPU-to-GPU communication with computation, essential in multi-GPU tensor-parallel setups; overlapping NCCL with compute can

recover 10–20%.

Memory Pooling / Fragmentation Control: Pre-reserves large GPU buffers for KV cache, shares blocks (vLLM/PagedAttention) to eliminate fragmentation waste, showing ~20% memory saving in the vLLM paper.

Other Low-Level Tricks: Hand-coded PTX/fused ops to reduce register-reuse overhead; casting to FP16/BF16 to use Tensor Cores; pinning threads to CPU sockets near the GPU (NUMA), sometimes worth ~5%.

Comparisons: Shared-memory fusion (FlashAttention-style) gives the single largest wins, multiplying GPU throughput several-fold. Asynchronous execution (streams, graphs) and memory pooling give smaller but vital improvements. SASS/PTX hand-tuning is the least-used, since compilers cover most needs.

Implementation & Hardware Details

Speculative Decoding: Realized by interleaving two forward passes; frameworks like vLLM or FasterTransformer enqueue both models, pipelining to hide the verification pass.

FlashAttention: Triton kernels (Python) or CUDA++ versions; simple drop-in replacement for standard scaled-dot-product-attention calls in PyTorch, or fused XLA ops in JAX/TensorFlow.

FlashInfer: Provides a runtime C++ library and code generation tools, linked for attention/GEMM, scheduling fused operators via JIT.

Torch.compile: Transparent to the user; profiling/caching handles compilation via FX traces, often pre-compiled ahead of serving.

AITemplate: Python front-end API similar to PyTorch, outputs a C++ library; integrates at build time so binaries contain kernels for each model.

GPU tricks: Often implemented in CUDA or Triton; libraries like Megatron-LM implement asynchronous KV-table updates via CUDA streams; C++ APIs use `cudaMemcpyAsync`, `cudaGraphLaunch`, etc.

Production Integrations

- **vLLM:** Uses Torch.compile, FlashAttention, PagedAttention, continuous batching, and CUDA graphs by default — a reference integration of many techniques.
- **NVIDIA Triton / FasterTransformer:** Offers spec-decoding (two-model) and FlashAttention; supports dynamic batching and CUDA graphs.
- **HuggingFace Transformers/TorchScript:** Torch.compile integration; fused kernels for GPU; Triton plugins for attention.
- **SGLang:** Uses FlashInfer, PagedAttention, etc. in its inference library.
- **TensorRT-LLM:** (proprietary) implements many fusions and multi-GPU pipelining.
- **Onnx-MLC/LLaMA.cpp:** Focus on memory (e.g. threaded WASM) more than kernel fusion.

Speculative decoding is just beginning to reach clouds broadly, but FlashAttention and Torch.compile are already default choices in many stacks.

Hidden and Experimental Methods

Less-publicized ideas highlighted: cluster-level primitives (ClusterFusion, exploiting Hopper DSMEM, not in any mainstream library yet); dynamic layer skipping (CLaSp, SPEED, requiring special training logic); speculative tree decodings (Traversals, new algorithms untested at scale); diffusion drafting (SpecDiff, relies on a new model class, hardly used in products); Flash-Sparse and NSA kernels (need sparse-trained models, niche); custom memory allocators (PyTorch caching vs alternatives like jemalloc for pinned memory); shape/time specialization (compiling models per-request shape to eliminate dynamic checks).

Comparative Summary

Area	Method	Speedup	Ease of Deploy	Robustness	Future Potential	Likely Standard
Speculative Decoding	Standard SpecDec	~2-3x	Moderate	High (exact)	Medium	Widely used
Speculative Decoding	Self-Spec (Draft&Verify)	~2x	Easy	High	Medium	Growing
Speculative Decoding	LayerSkip/Early-Exit	~2x	Hard	Medium	Medium	Niche
Speculative Decoding	Lookahead	1.8-4x	Hard	Medium	High	Research
Speculative Decoding	SpecDiffusion	~7x	Hard	Unknown	High	Uncertain
Attention Kernels	FlashAtt v1/v2	5-9x; 10-20x mem	Easy	High	Low (mature)	Standard
Attention Kernels	FlashAtt-3/4	2-3x vs prior	Moderate	High	High	Niche/specialty
Attention Kernels	FlashInfer	13-69% lat reduction	Moderate	High	High	Emerging
Attention Kernels	AITemplate	up to 12x	Hard	High	High	Niche (Meta use)
Attention Kernels	FSA (sparse)	1.6x kernel	Hard	Medium	Medium	Research
Attention Kernels	ClusterFusion	1.61x latency	Very hard	Unproven	High	Unlikely soon
Scheduling	Continuous (vLLM)	~3-4x	Hard	High	High	Likely dominant
Scheduling	Multi-step	+28% on 70B	Moderate	High	Medium	Seen in vLLM
Compiler	Torch.compile	1.8-2x	Very easy	High	Low	Mainstream
Compiler	AITemplate	up to 12x	Hard	High	High	Select use
Memory/Exec	FlashAtt fusion	~3.8x (layer ex.)	Hard	High	Low	Standard
Memory/Exec	Paged KV (vLLM)	2-4x throughput	Hard	High	High	Emerging (vLLM)

Stability and Obscurity Rankings

Speculative Decoding: Standard SpecDec — stable, known to specialists. Self-Spec — stable, known. LayerSkip — experimental, niche. Lookahead — prototype, niche. SpecDiffusion — prototype, deep underground. DART — prototype, niche. JetSpec — prototype, nearly invisible. Traversal Verify — research, known to specialists.

Attention Kernels: FlashAttention 1/2 — production-proven, mainstream. FlashAttention 3/4 — production (v3) / prototype (v4), specialists. FlashInfer — stable,

specialists. AITemplate — stable, specialists. SageAttn — prototype, known to specialists. PagedAttention — stable, known to specialists. FSA/Flash-Sparse — prototype, niche. ClusterFusion — prototype, deep underground.

Scheduling/Batching: Static/Dynamic Batch — stable, mainstream. Continuous (vLLM) — stable, known to specialists. Token-level scheduling — prototype, deep underground. Chunked prefill/multi-step — stable, known to specialists. Fairness/priorities — experimental, niche.

Compiler/Graph: Torch.compile — stable, mainstream. TensorRT — stable, mainstream (NVIDIA stack). AITemplate — stable, specialists. XLA/MLIR/TVM — mature in parts, specialists/non-mainstream. Graph rewriting — stable, mainstream practice.

Memory/Execution: Shared-memory fusion — production, mainstream. Paged KV (vLLM) — stable, known. CUDA Graphs/Streams — production, mainstream. Memory pooling — production, known. PTX/ASM tricks — deep underground.

Research Lineage Maps

Speculative Decoding Map: Traditional decoding → Speculative Sampling (Chen 2023) → Leviathan SpecDec (2022) → Self-Spec (Zhang 2023) → LayerSkip (Meta 2024), CLaSp (2025) → SpecPV (2025) → Lookahead (2024) → Diffusion drafts (DART 2024, SpecDiff 2025) → Cluster/tree decoders (RSD 2024, JetSpec 2026, Traversal 2025). Future likely merges diffusion with tree verification.

Attention Kernel Map: Naive attention → FlashAttention (2022) → FlashA-2,3 (2023–24) → SageAttn quantized (2024–25) → FlashA-4, ClusterFusion (2025–26). Sparse pipelines: BigBird/NSA → Flash-Sparse (2026). AITemplate (2022) fused more ops in parallel; FlashInfer (2025) integrated with scheduling.

Scheduling Map: Simple batching → Triton dynamic batching → Continuous/vLLM batching (2023) → Token-level (FlexLLM 2024). PagedAttention and multi-step scheduling emerged 2023–24. Future: hybrid cloud-edge scheduling and finer LLM “kernel” scheduling.

Compiler Map: Static graph (TorchScript) → Torch.compile (Inductor) (2023) → Fusion frameworks (AITemplate 2022, TensorRT) → MLIR-based (OpenXLA, IREE). PyTorch 2.x consolidated many features; future likely brings more dynamic graph compilers and domain-specific IRs.

Memory/Exec Map: GPU mem pooling → Paged KV (2023) → Unified KV sharing (2024). Overlap (streams) is old; CUDA Graphs for LLM specifically (2023+). ClusterFusion (2025) uses new GPU features; upcoming GPU features expected to open more tricks.

Future Directions

- **Unified Execution Engine:** Combining many techniques — an inference engine that automatically applies Torch.compile and speculative decoding with optimal draft selection per query.
- **Adaptive Hybrid Strategies:** Dynamically switching between methods per workload (e.g. spec-decoding only if acceptance likely).
- **LLM-Designed Hardware Blocks:** Hardware primitives for speculative parallelism (future GPUs might expose an “LLM shader”).

- **Memory-Compute Co-design:** Closer integration of cluster-level comms (like ClusterFusion) into CUDA or Transformer libraries.
- **Differentiable Compilers:** Auto-tuning by RL or genetic algorithms to find best optimizations for a given model.
- **Streaming / Edge Inference:** Applying these techniques at small batch sizes or on smaller devices — hybrid CPU/GPU schedules or even FPGAs with custom streams.

Comprehensive Bibliography

Selected key sources referenced throughout this report include: Leviathan et al., “Fast Inference from Transformers via Speculative Decoding,” ICLR 2023; Chen, J.X., et al., “Speculative Sampling,” NAACL 2023; Zhang, J., et al., “Draft & Verify,” ACL 2024; Tan, W., et al., “SpecPV,” ICML 2025; Fu, Z., et al., “Lookahead Decoding,” arXiv 2024; Christopher, C., et al., “Speculative Diffusion Decoding,” NAACL 2025; Liu, Z., et al., “DART,” ICML 2024; Luo, X., et al., “ClusterFusion,” NeurIPS 2025; Dao, T., et al., “FlashAttention” series, NeurIPS 2022 / ICLR 2024 / arXiv 2026; Ye, Z., et al., “FlashInfer,” MLSys 2025; Kwon, W., et al., “Efficient Memory Management for LLM Serving with PagedAttention,” SOSP 2023; Yan, R., et al., “FSA,” ICLR 2026; Meta AI, “AITemplate,” blog 2022; HuggingFace/vLLM engineering blogs on Torch.compile and performance updates.

LocalHouseLLM Research — Internal Technical Compendium

Document prepared by LocalHouseLLM Research. All findings, ratings, and citations herein reflect internal review of primary and secondary sources current as of the report date. For internal / partner distribution.